CEREBRO Solution Architecture

Architecture · CEREBRO · Joselito Dizon · 2026-08-02

Containers, data stores and the staged plan from authentication to knowledge graph, reasoning and visualisation.

Context

This architecture extends the Vision and Concept Note into a buildable system.

It applies the retrieval and graph patterns covered in the Knowledge Graphs and RAG lecture notes.

Containers

The web tier is Next.js with React; the API is FastAPI with SQLAlchemy; PostgreSQL holds users, artifacts and fragments.

React never connects to PostgreSQL directly: all data access goes through the API, and the web pod proxies requests under one origin.

Later stages add an object store for originals, a vector store for embeddings, a graph store for entities and relationships, and a local LLM served by Ollama.

Pipeline

An artifact moves through upload, file inspection, metadata extraction, AI enrichment, human review and registration.

Registered artifacts are segmented into knowledge fragments, each carrying concepts, entities, temporal fields and provenance.

Every pipeline step emits telemetry so the user can see which stage is running and how long it took.

Deployment

Kubernetes in a dedicated cerebro namespace is the primary target, with Docker Compose as the simpler fallback.

Host ports are configurable because ports 3000, 8000 and 5432 are often taken on a shared machine.